

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

AI and Application Development and Jira

M1:Practical – VI

Roll No.: T002	Name: Asmitha Mohan Raj
Class: TYIT	Batch: 1
Date of Assignment: 19/08/2026	Date/Time of Submission: 19/08/2026

Probability Reasoning (Bayesian Networks & Inference)

1. Using a simple example (Burglary–Alarm), create a Bayesian Network using pgmpy.

Code & Output:

```
from pgmpy.models import
DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])
# Step 2: Define probability of Burglary
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)
# Define probability of Earthquake
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)
# P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)
```

```

    }
)
# P(JohnCalls | Alarm)
cpd_john = TabularCPD(
    variable="JohnCalls",
    variable_card=2,
    values=[
        [0.95, 0.10], # JohnCalls = False
        [0.05, 0.90] # JohnCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "JohnCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)
# P(MaryCalls | Alarm)
cpd_mary = TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99, 0.30], # MaryCalls = False
        [0.01, 0.70] # MaryCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "MaryCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)
# Step 3: Add CPDs to model
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm,
    cpd_john,
    cpd_mary
)
```

```
)
# Step 4: Check model correctness
print("Is Bayesian Network valid?",
model.check_model())
# Step 5: Perform inference
inference = VariableElimination(model)
result = inference.query(
    variables=["Burglary"],
```

```
Is Bayesian Network valid? True
```

```
Probability of Burglary given JohnCalls=True and MaryCalls=True:
```

```
+-----+-----+
| Burglary | phi(Burglary) |
+-----+-----+
| Burglary(False) | 0.7158 |
+-----+-----+
| Burglary(True) | 0.2842 |
+-----+-----+
```

```
evidence={
    "JohnCalls": "True",
    "MaryCalls": "True"
}
)
print("\nProbability of Burglary given
JohnCalls=True and MaryCalls=True:")
print(result)
```

A.Example:

Code & Output:

```
from pgmpy.models import
DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Power", "Internet"),
    ("Router", "Internet")
])
# Step 2: Define probability of Power
cpd_power = TabularCPD(
    variable="Power",
    variable_card=2,
    values=[[0.10], [0.90]],
    state_names={"Power": ["False", "True"]}
)
# Define probability of Router
cpd_router = TabularCPD(
    variable="Router",
    variable_card=2,
    values=[[0.20], [0.80]],
    state_names={"Router": ["False", "True"]}
)
# P(Internet | Power, Router)
cpd_internet = TabularCPD(
    variable="Internet",
    variable_card=2,
    values=[
        [0.99, 0.80, 0.70, 0.05], # Internet = False
        [0.01, 0.20, 0.30, 0.95] # Internet = True
```

```
],
evidence=["Power", "Router"],
evidence_card=[2, 2],
state_names={
    "Internet": ["False", "True"],
    "Power": ["False", "True"],
    "Router": ["False", "True"]
}
)
# Step 3: Add CPDs to model
model.add_cpds(
    cpd_power,
    cpd_router,
    cpd_internet
)
# Step 4: Check model correctness
print("Is Bayesian Network valid?",
model.check_model())
# Step 5: Perform inference
inference = VariableElimination(model)
result = inference.query(
    variables=["Internet"],
    evidence={
        "Power": "True",
        "Router": "True"
    }
)
print("\nProbability of Internet given Power=True
and Router=True:")
print(result)
```

```

Is Bayesian Network valid? True

Probability of Internet given Power=True and Router=True:
+-----+-----+
| Internet | phi(Internet) |
+-----+-----+
| Internet(False) | 0.0500 |
+-----+-----+
| Internet(True) | 0.9500 |
+-----+-----+

```

2. Write a program to compute Posterior Probability using Bayesian Networks(Burglar|Alarm = True)

Code & Output:

```

# Posterior Probability using Bayesian Network
# Query: P(Burglary | Alarm = True)
from pgmpy.models import
DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm")
])
# Step 2: Define probability of Burglary
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]})
# Step 3: Define probability of Earthquake
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]})
# Step 4: Define probability of Alarm
# P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,

```

```

    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)
# Step 5: Add CPDs to model
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm
)
# Step 6: Check whether model is correct
print("Is Bayesian Network valid?",
model.check_model())
# Step 7: Perform inference
inference = VariableElimination(model)
result = inference.query(
    variables=["Burglary"],
    evidence={"Alarm": "True"}
)
print("\nPosterior Probability:")
print("P(Burglary | Alarm = True)")
print(result)

```

```

Is Bayesian Network valid? True

Posterior Probability:
P(Burglary | Alarm = True)
+-----+-----+
| Burglary | phi(Burglary) |
+-----+-----+
| Burglary(False) | 0.6264 |
+-----+-----+
| Burglary(True) | 0.3736 |
+-----+-----+

```

B.Example:**Code & Output:**

```
# Posterior Probability using Bayesian Network
# Query: P(Connection | WiFi = True)
from pgmpy.models import
DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Internet", "Connection"),
    ("WiFi", "Connection")
])
# Step 2: Define probability of Internet
cpd_internet = TabularCPD(
    variable="Internet",
    variable_card=2,
    values=[[0.10], [0.90]],
    state_names={"Internet": ["False", "True"]}
)
# Step 3: Define probability of WiFi
cpd_wifi = TabularCPD(
    variable="WiFi",
    variable_card=2,
    values=[[0.20], [0.80]],
    state_names={"WiFi": ["False", "True"]}
)
# Step 4: Define probability of Connection
# P(Connection | Internet, WiFi)

cpd_connection = TabularCPD(
    variable="Connection",
    variable_card=2,
    values=[
        [0.99, 0.70, 0.80, 0.10], # Connection = False
```

```
        [0.01, 0.30, 0.20, 0.90] # Connection = True
    ],
    evidence=["Internet", "WiFi"],
    evidence_card=[2, 2],
    state_names={
        "Connection": ["False", "True"],
        "Internet": ["False", "True"],
        "WiFi": ["False", "True"]
    }
)
# Step 5: Add CPDs to model
model.add_cpds(
    cpd_internet,
    cpd_wifi,
    cpd_connection
)
# Step 6: Check whether model is correct
print("Is Bayesian Network valid?",
      model.check_model())

# Step 7: Perform inference
inference = VariableElimination(model)

result = inference.query(
    variables=["Connection"],
    evidence={"WiFi": "True"}
)
print("\nPosterior Probability:")
print("P(Connection | WiFi = True)")
print(result)
```

```
Is Bayesian Network valid? True
```

```
Posterior Probability:
```

```
P(Connection | WiFi = True)
```

```
+-----+-----+
| Connection | phi(Connection) |
+-----+-----+
| Connection(False) | 0.1600 |
+-----+-----+
| Connection(True) | 0.8400 |
+-----+-----+
```